Chapter 4
Retrieval-Augmented Generation

Vector search, vector databases, hybrid retrieval, re-ranking and multimodal RAG



01

Why RAG

What problem are we solving?



LLMs alone are not enough

Frozen knowledge

A model only knows what was in its training data. Anything newer simply does not exist for it.

No private data

Your company files, course notes or product manuals were never part of training.

Confident guessing

When the model does not know, it often makes up a fluent, wrong answer: a hallucination.

RAG fixes all three by handing the model the right facts at the moment it answers.

RAG is an open-book exam

Closed-book

A plain LLM answers from memory. If it never learned the fact, it guesses.

Open-book

A RAG system first finds the right pages, then writes the answer using them.

1

Retrieve

Search the documents for the chunks most related to the question.

2

Augment

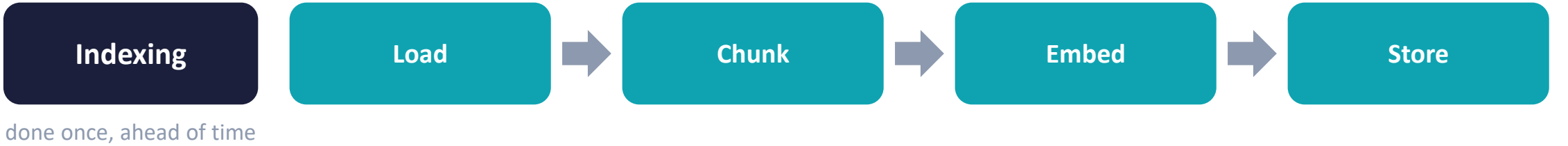
Paste those chunks into the prompt as context.

3

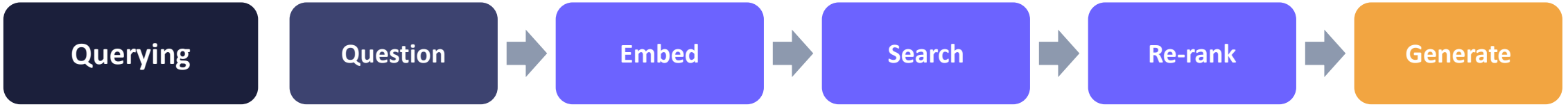
Generate

The LLM answers using only that context.

The two halves of every RAG system



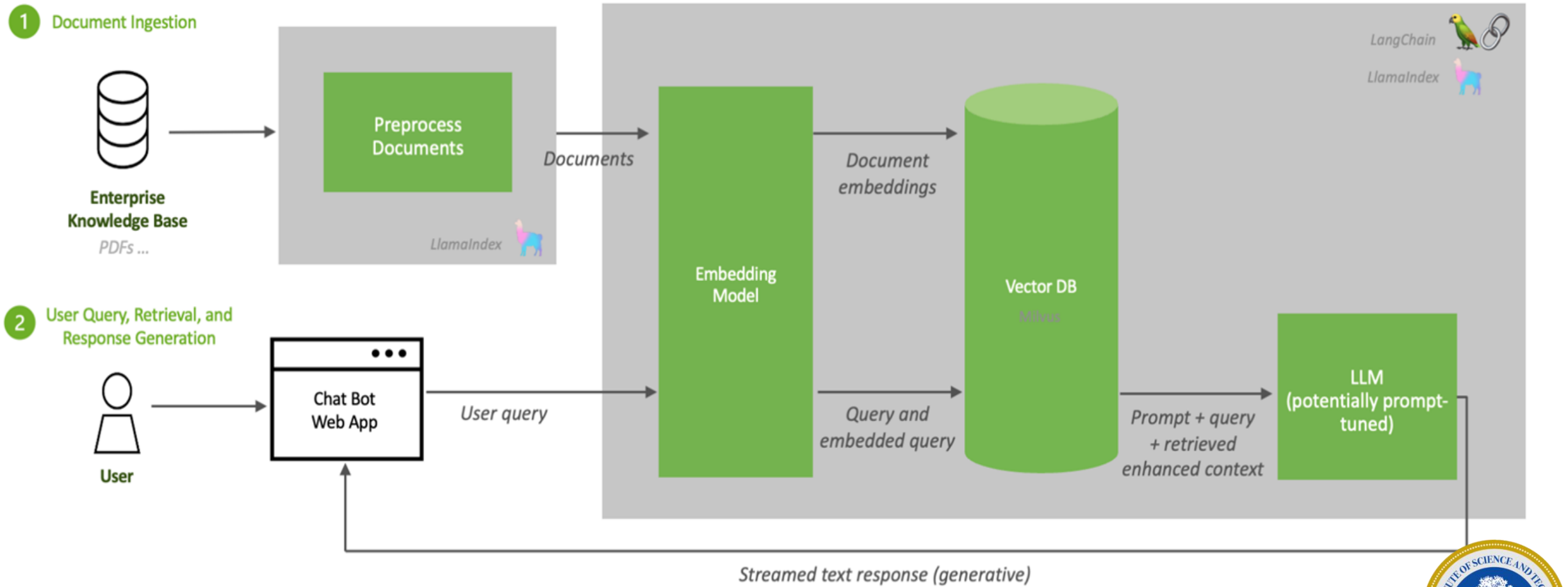
done once, ahead of time



every question

The vector store built during indexing is what the search step looks into.

Retrieval Augmented Generation (RAG) Sequence Diagram



02

Preparing the data

Loading, embeddings and chunking



Step 1: load and clean the documents

Text

Read page by page with pdfplumber: page.extract_text()

Tables

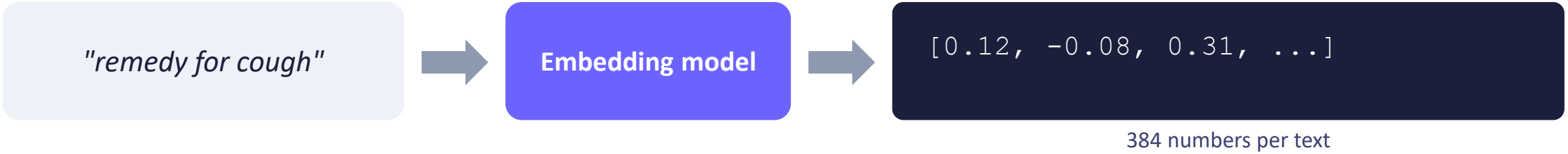
From CSV or Excel with pandas, or from a PDF with page.extract_tables()

Images

Charts, diagrams and photos. We make them searchable later with captions.

Cleaning matters: odd bullet symbols, lines broken mid-sentence, page headers and footers all end up inside your chunks and weaken search.

Embeddings turn meaning into numbers



Similar meaning gives vectors that point the same way



Each compared with "remedy for cough". With normalised vectors, the dot product equals cosine similarity: 1 means same meaning, around 0 means unrelated.

Why we split documents into chunks

Chunks too large

- One vector has to represent many topics, so it matches nothing well
- Irrelevant text fills the prompt and costs tokens
- Embedding models truncate long inputs

Chunks too small

- A sentence loses the context around it
- The answer gets split across several chunks
- More chunks to store and search



Aim for one idea per chunk: big enough to make sense alone, small enough to stay on one topic.

Four ways to chunk

Fixed-size

Cut every N characters, with some overlap.

Simplest baseline; can cut words and sentences in half.

Recursive

Try paragraphs, then lines, then sentences, then words, until pieces fit.

Best general-purpose default.

Section-based

Split on the document's own headings.

Ideal for manuals and guides with clear structure.

Semantic

Start a new chunk when the meaning of the next sentence changes.

For text with no clear structure; needs a threshold.

Same document, different chunks

Fixed-size (300 characters)

```
...2.1 Cold & Flu
- Ginger Tea: Boil ginger
  slices in water, add honey
  and lemon...
- Turmeric Milk: Mix 1 tsp
  turmeric in warm milk. Acts
  as an anti-inflammatory a
```

The chunk ends in the middle of a word.

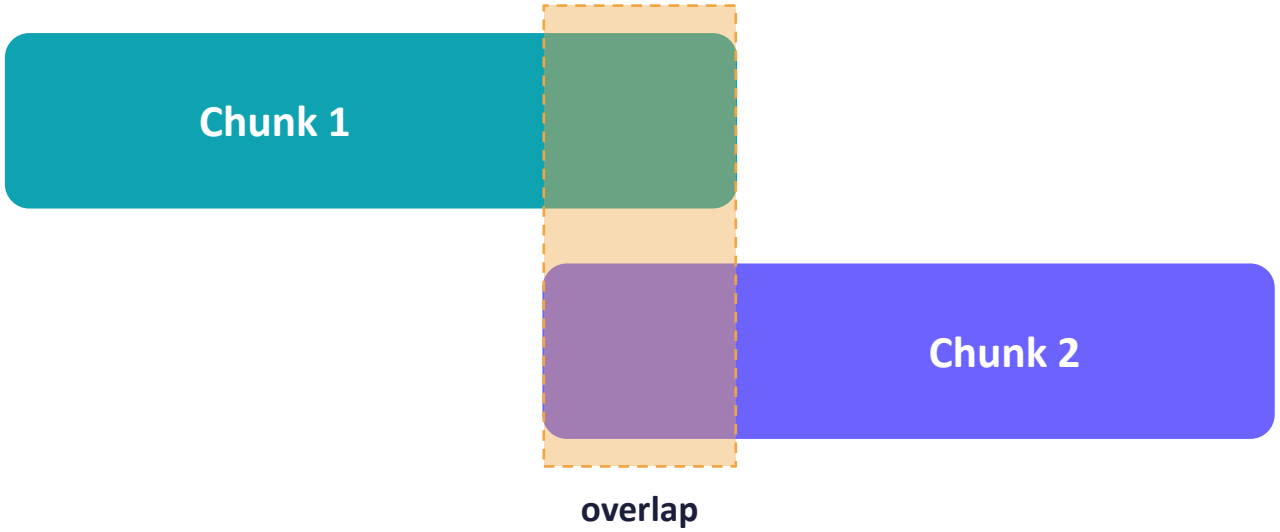
Section-based

```
2.2 Cough
- Honey & Lemon: Mix 1 tbsp
  honey with warm water and
  lemon juice. Soothes the
  throat and reduces
  coughing.
- ...
```

One complete topic, heading included.

On the sample guide: fixed-size gives 15 chunks, recursive 15 and section-based 16. Similar counts, very different quality.

Chunk size and overlap



Size

Maximum characters (or tokens) per chunk. 200 to 500 characters is a good starting range.

Overlap

Repeats the end of one chunk at the start of the next, so an idea cut at a boundary still appears whole somewhere. Around 10 to 20% of the size.

Tune both by asking real questions and reading what comes back.

03

Vector search and basic RAG

Finding the right chunks, then answering

Vector search in three steps

1 Embed every chunk

Once, during indexing. Keep the vectors in a matrix.

2 Embed the question

Using the same embedding model.

3 Score and sort

Dot product with every chunk vector, keep the top k.

```
class SimpleVectorStore:
    def __init__(self, chunks):
        self.chunks = chunks
        self.vectors = embed([c["text"] for c in chunks])    # step 1

    def search(self, query, k=3):
        query_vector = embed([query])[0]                     # step 2
        scores = self.vectors @ query_vector                 # step 3
        best = np.argsort(scores)[::-1][:k]
        return [(self.chunks[i], float(scores[i])) for i in best]
```



Augment: building the prompt

```
You are a helpful assistant.
Answer the question using ONLY
the context below.
If the answer is not in the
context, say "I don't know".
Keep the answer short.

Context:
{context}

Question: {question}
Answer:
```



Grounding

"ONLY the context" keeps the answer tied to your documents.



Permission to refuse

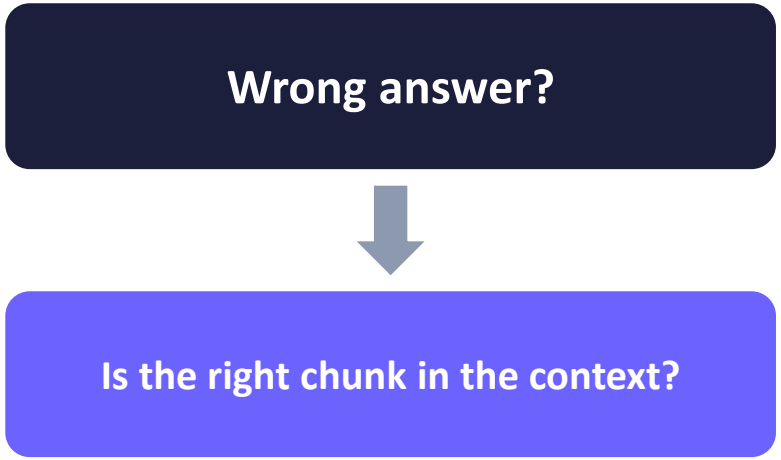
"Say I don't know" sharply reduces made-up answers.



Context slot

The retrieved chunks are joined and pasted here.

Always look at what was retrieved



No → retrieval problem

Change the chunking, increase k, try hybrid search or a re-ranker.

Yes → generation problem

Improve the prompt, put the best chunk first, or use a stronger model.

04

Vector stores and databases

FAISS, Qdrant and Pinecone



From a NumPy array to a database



Speed at scale

Comparing with every vector is fine for 20 chunks, not for 20 million.



Updates and deletes

Add or remove documents without rebuilding everything.



Persistence

Our store disappears when Python stops.



Many users

Serve many queries at once from a server.



Metadata filtering

Search only chunks where type = table, or year = 2024.

Exact vs approximate search

Flat (exact)

Compare the query with every vector. Perfect accuracy; slow when the collection is huge.

IVF (clusters)

Group vectors into clusters, then search only the clusters nearest to the query.

HNSW (graph)

Link each vector to its neighbours and hop through the graph toward the query.

Approximate nearest neighbour (ANN) indexes trade a tiny amount of accuracy for a huge gain in speed. Qdrant uses HNSW by default.

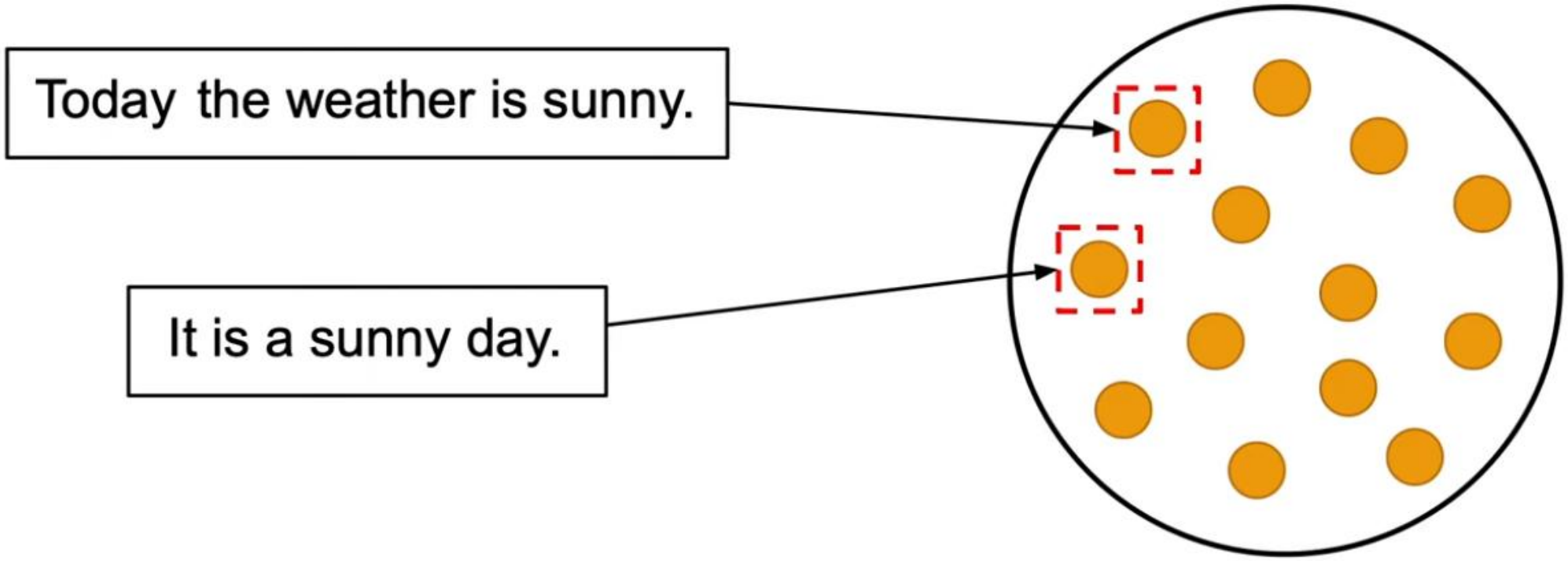
Vector Databases: The Super-Fast Library for Vectors

Analogy: Imagine a giant library with millions of books.

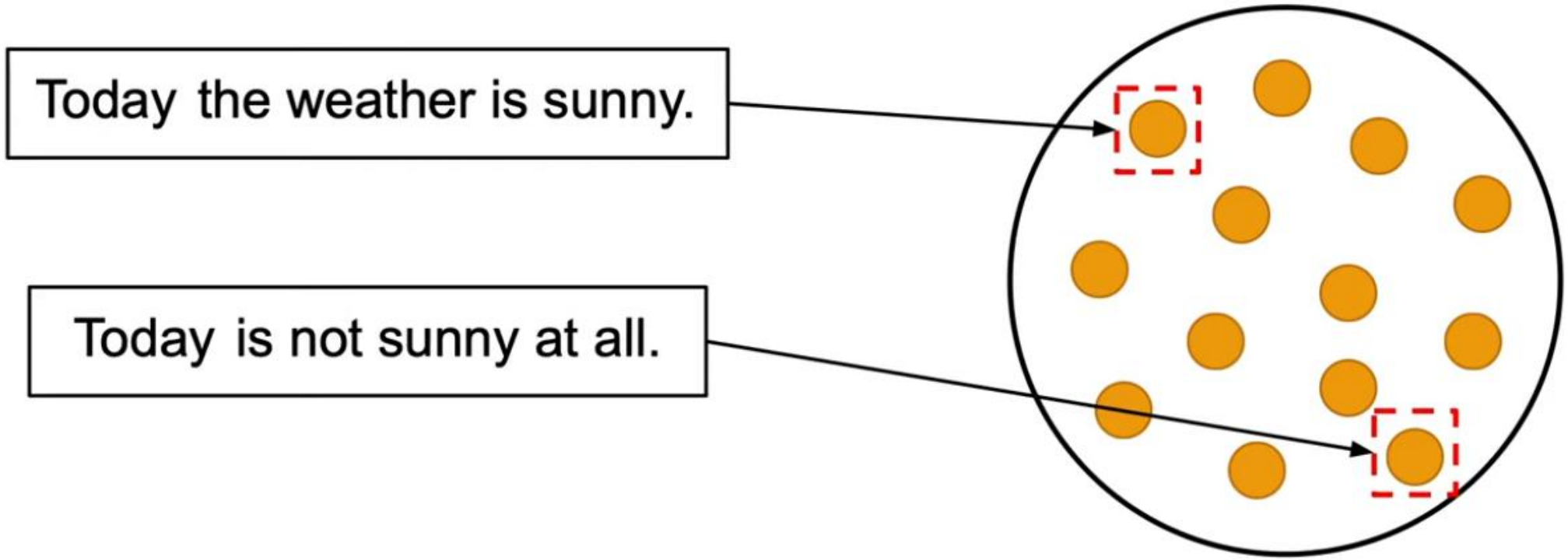
A normal database (slow way): To find a book about "ancient Roman history," you'd have to read the title of every single book in the library.

A vector database (fast way): The librarian has already organized the books into sections based on their topic. All history books are in one wing, all science in another. When you ask for "ancient Roman history," the librarian doesn't check the science section at all. They go directly to the history wing and then to the "classical history" aisle to find the closest matches.

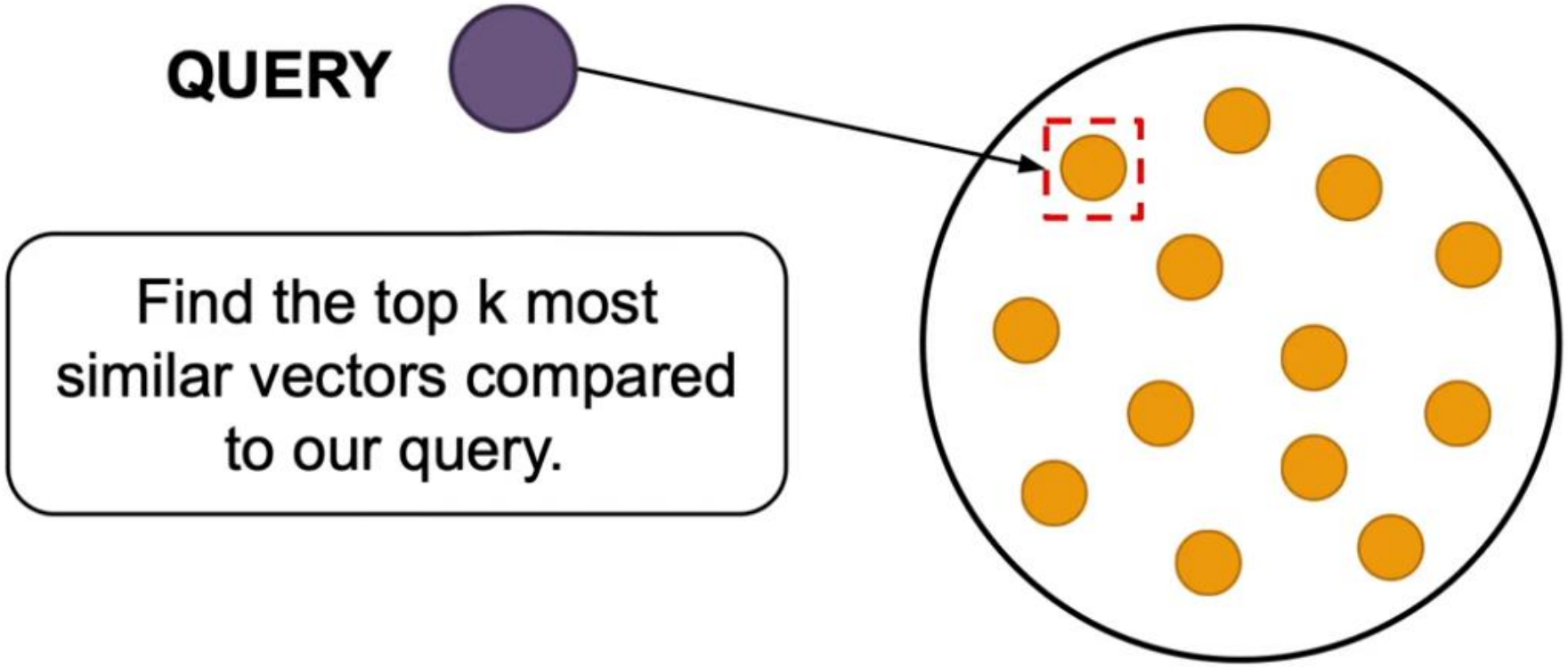
Hierarchical Navigable Small Worlds (HNSW)



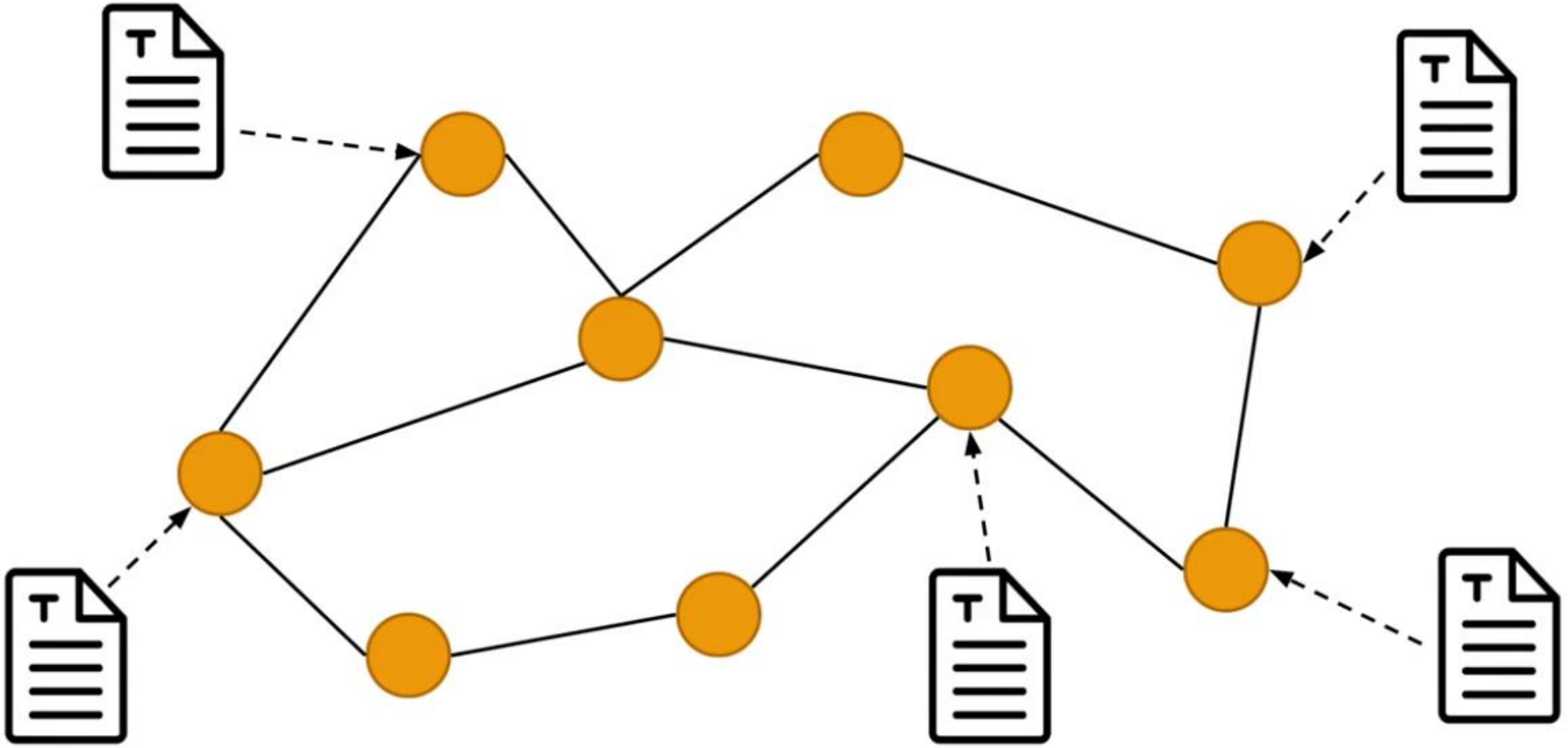
Hierarchical Navigable Small Worlds (HNSW)



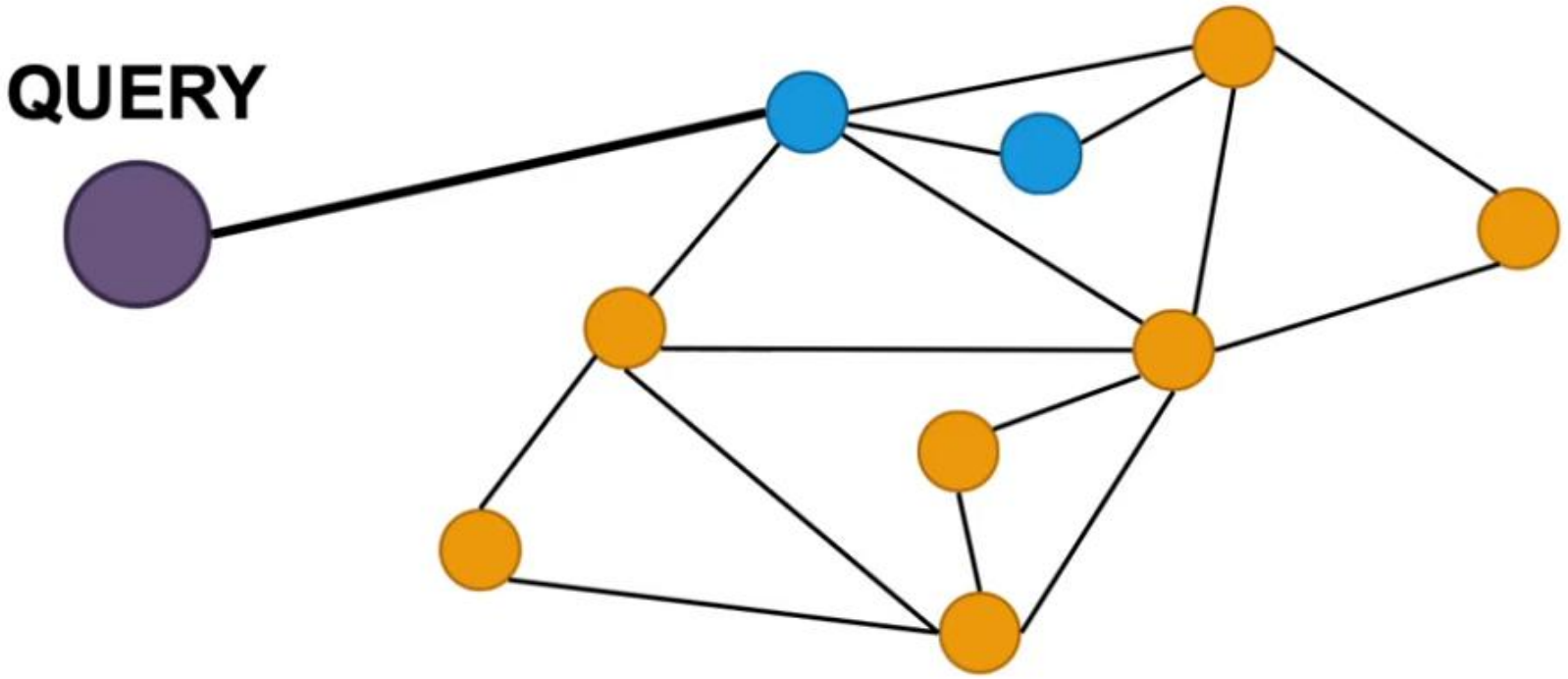
Hierarchical Navigable Small Worlds (HNSW)



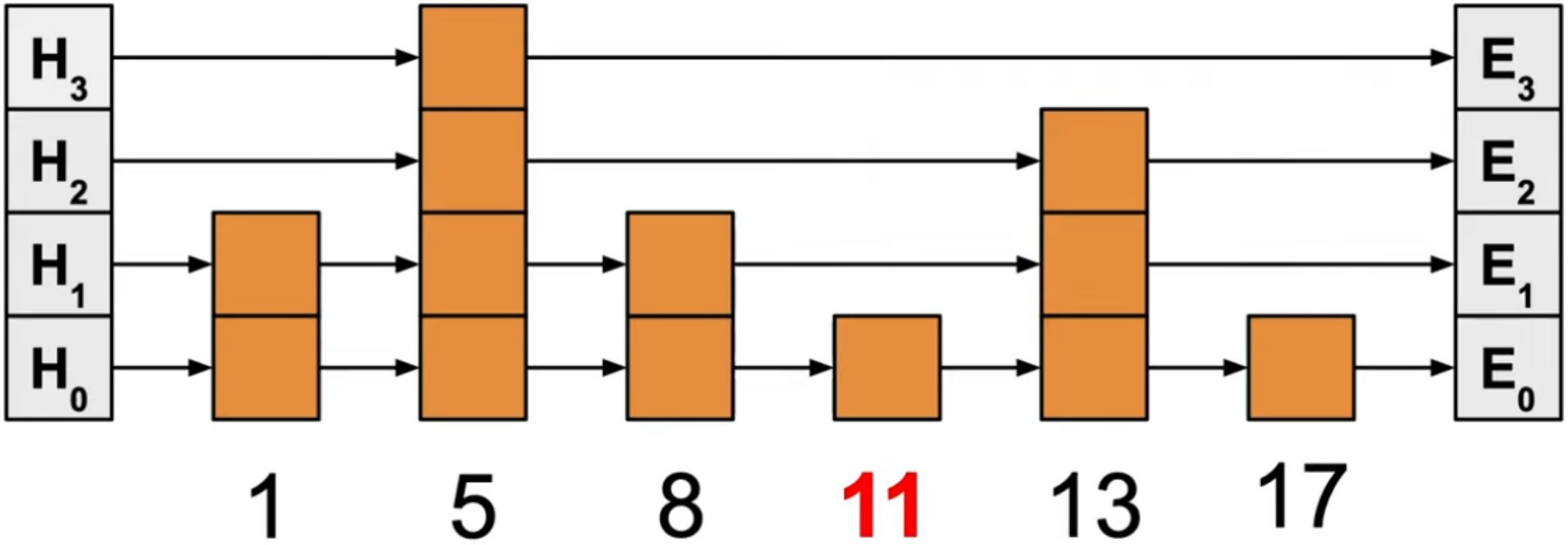
Hierarchical Navigable Small Worlds (HNSW)



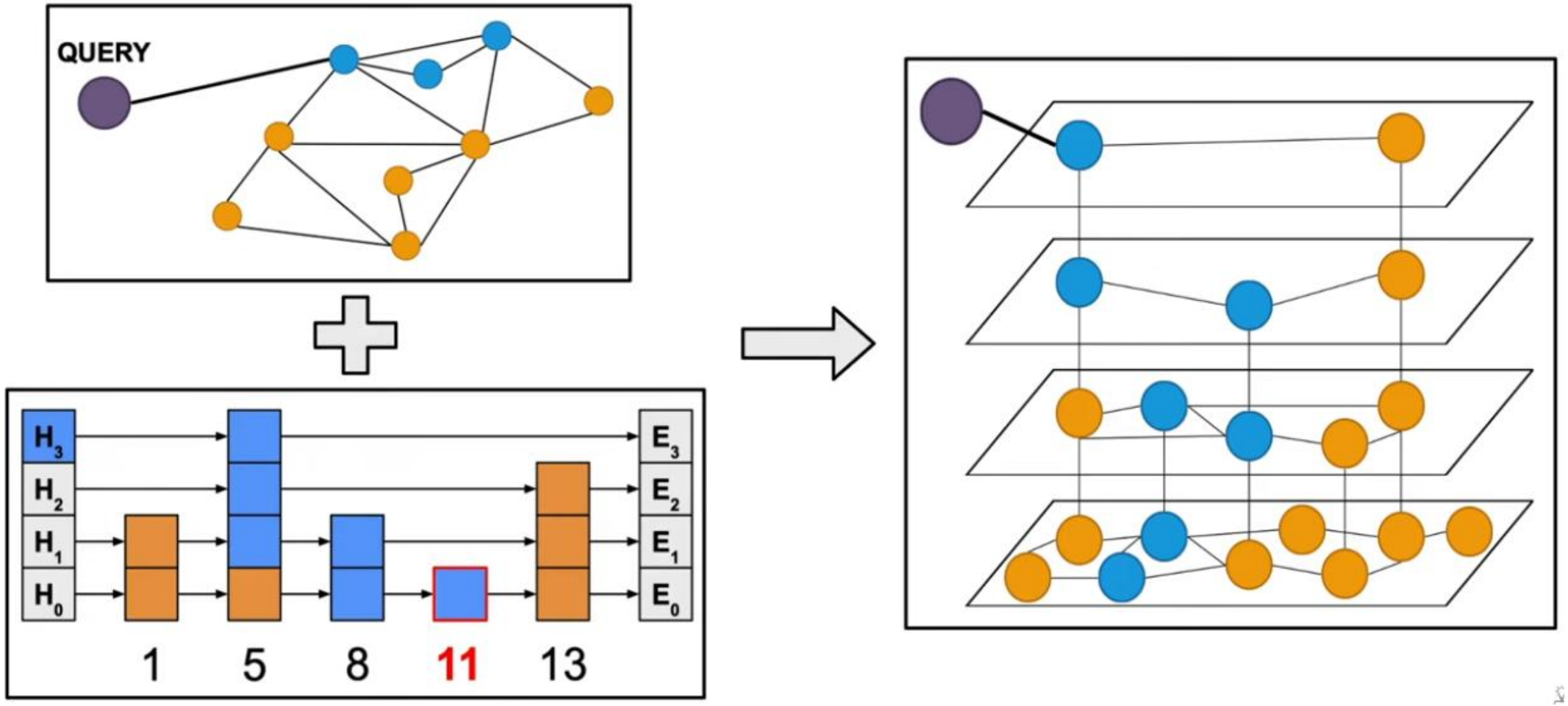
Hierarchical Navigable Small Worlds (HNSW)



Hierarchical Navigable Small Worlds (HNSW)



Hierarchical Navigable Small Worlds (HNSW)



Top VectorStores

#1 Chroma

#2 FAISS

#3 Pinecone

#4 qdrant

#5 docarray

#6 weaviate

#7 PostgreSQL

#8 supabase

#9 neo4j

#10 redis

#11 Azure Cognitive Search

#12 Astra DB



FAISS: a vector search library

- Open-source library from Meta
- Runs inside your Python process; nothing to install or host
- Very fast, with many index types (flat, IVF, HNSW)
- Stores only vectors: keep your chunk text in a Python list
- Save to disk manually with faiss.write_index

```
import faiss

index = faiss.IndexFlatIP(384)
index.add(chunk_vectors)

scores, ids = index.search(
    embed([query]), k=3)

results = [chunks[i] for i in ids[0]]
```

IP = inner product, which equals cosine similarity for normalised vectors.

Qdrant: open-source vector database

Collection like a table

Point one row: id + vector + payload

Payload metadata: text, type, source

1 In-memory

No setup. Runs inside Python; ideal for class and Colab.

```
QdrantClient(":memory:")
```

2 Docker

Local server with a dashboard at localhost:6333.

```
docker run -p 6333:6333
  qdrant/qdrant
```

3 Qdrant Cloud

Free cluster at cloud.qdrant.io; data persists online.

```
QdrantClient(url=...,
  api_key=...)
```

Same code for all three: create_collection → upsert → query_points.



Pinecone: fully managed vector database

1

Sign up

Free Starter plan at app.pinecone.io

2

Copy the API key

Save it as PINECONE_API_KEY in Colab Secrets

3

Create a serverless index

dimension = 384, metric = cosine

4

Upsert and query

Vectors with metadata in, top-k matches out

```
pc = Pinecone(api_key=KEY)
pc.create_index(name="rag-demo",
    dimension=384, metric="cosine",
    spec=ServerlessSpec(
        cloud="aws",
        region="us-east-1"))

index = pc.Index("rag-demo")
index.upsert(vectors=[...])
index.query(vector=q, top_k=3,
    include_metadata=True)
```

New vectors take a few seconds to become searchable.

Choosing a vector store

	FAISS	Qdrant	Pinecone
Type	Library	Open-source database	Managed cloud database
Runs where	Inside your program	In-memory, Docker or cloud	Pinecone's cloud only
Persistence	Manual save to file	Yes (Docker or cloud)	Yes
Metadata filters	No	Yes	Yes
Best for	Experiments and research	Self-hosted production, local work	Production with zero infrastructure

All three return the same top results here, because they search the same vectors.



05

Keyword and hybrid search

BM25, and combining it with vectors



BM25: classic keyword search

Term frequency

A word that appears more often in a chunk raises its score, with diminishing returns.

Rarity (IDF)

Rare words such as "Epsom" count far more than common words such as "the".

Length normalisation

Long chunks do not win just because they contain more words.

Meaning vs exact words

Query

"Epsom salt"

BM25 wins

A rare exact term. BM25 scores the one chunk that contains it far above everything else.

Also: product codes, names, acronyms, error messages.

Query

"trouble falling asleep"

Vector search wins

Not one of these words appears in the guide, so BM25 scores every chunk zero. Vector search still finds the Insomnia section.

Also: paraphrases, synonyms, questions in everyday language.

Hybrid search with Reciprocal Rank Fusion

```
RRF score = 1 / (60 + rank in vector list)  +  1 / (60 + rank in BM25 list)
```

Chunk	Vector rank	BM25 rank	RRF score
A	1	3	1/61 + 1/63 = 0.0323
B	2	1	1/62 + 1/61 = 0.0325
C	3	not found	1/63 = 0.0159

B wins

Strong in both lists beats being first in only one.

Uses only ranks, so BM25 scores and cosine scores never need to be compared. The constant 60 stops the top rank from dominating.

Weighted fusion

```
score = alpha × vector_score  +  (1 − alpha) × bm25_score
```



alpha = 0

pure BM25

alpha = 0.5

equal mix

alpha = 1

pure vector

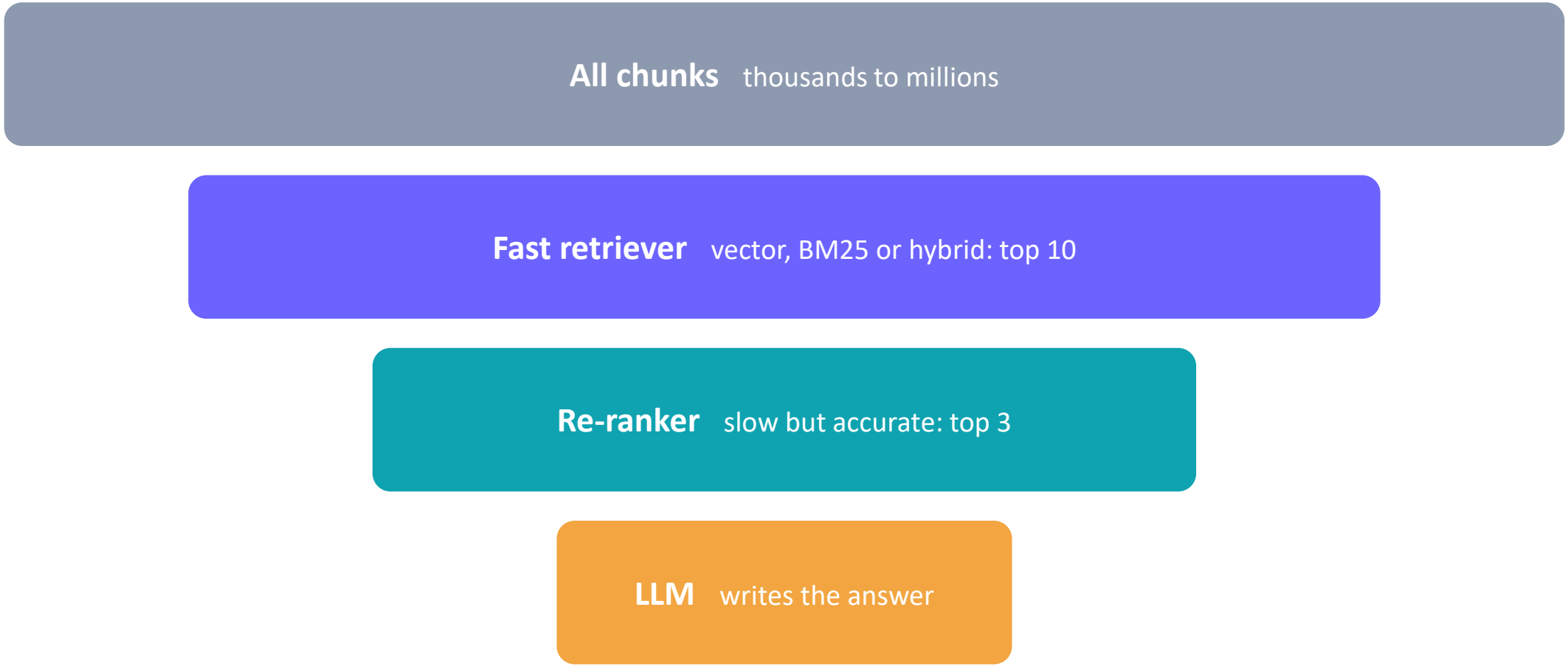
Scale first: BM25 scores can be 0 to 10 or more, cosine scores are about 0 to 1. Min-max scaling puts both on 0 to 1 before mixing, otherwise BM25 would drown out vector search.

06

Re-ranking

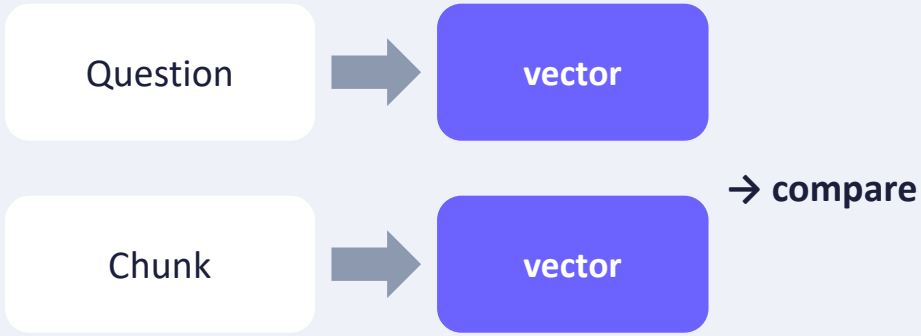
Fast retrieval first, careful ranking second

Two-stage retrieval



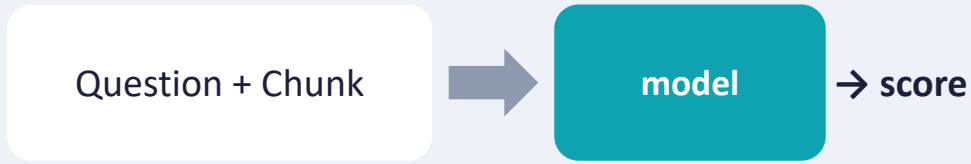
Bi-encoder vs cross-encoder

Bi-encoder (vector search)



Fast: chunk vectors are computed once, ahead of time. But question and chunk never "see" each other.

Cross-encoder (re-ranker)



Accurate: reads both texts together. But it must run once per pair, so only on a few candidates.

The LLM as a re-ranker

```
On a scale of 0 to 10, how useful is this passage
for answering the question?
Question: {query}   Passage: {chunk}
Reply with only the number.
```

Pros

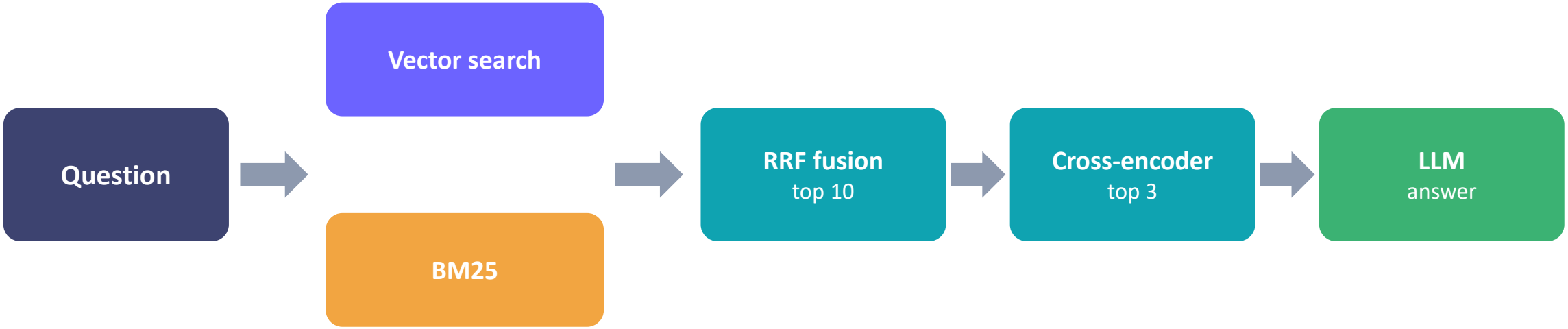
- No extra model to download
- Understands nuance and instructions
- Easy to explain and customise

Cons

- One API call per candidate: slow
- Costs tokens and hits rate limits
- Replies must be parsed into numbers



The complete advanced pipeline




```
def advanced_search(query, k=3):
    candidates = hybrid_rrf_search(query, k=10)
    return rerank(query, candidates, top_n=k)
```

07

Multimodal RAG

Text, tables and images in one system

Real documents are not just text

Text

Paragraphs and lists. Handled by everything we have built so far.

Tables

Exact numbers in rows and columns. Plain text extraction often scrambles them.

Images and charts

Information that exists only as pixels. A text-only pipeline never sees it.

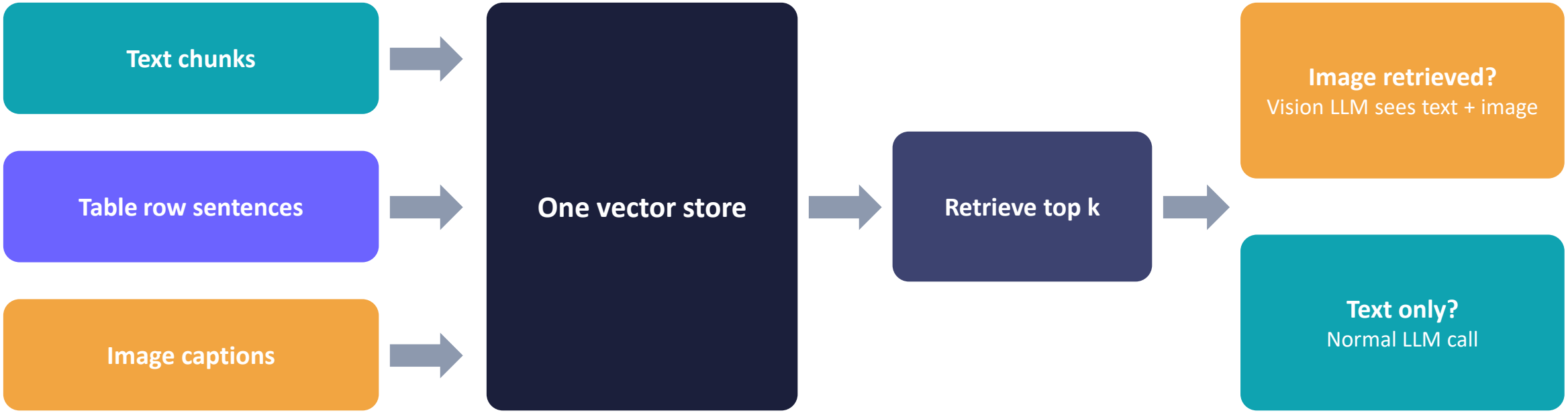
If a fact lives only in a table or chart, a text-only RAG system will answer "I don't know".

Turn everything into searchable text

Content	Convert to	Example
Text	Chunks, as before	2.2 Cough - Honey & Lemon: Mix 1 tbsp honey...
Table	One sentence per row, plus the whole table as one chunk	On 2023-01-05 the North region sold 10 units of Laptop for a sale amount of 12000.
Image	A caption written by a vision LLM	Bar chart of total sale amount by region: North, South, East and West...

At answer time, if an image is retrieved, the actual image is sent to the vision LLM too.

The multimodal RAG pipeline



Each chunk keeps a type (text, table or image) and, for images, the file path. Try:

"Which region had the highest total sales?" *"How many keyboards were sold in the East?"* *"Home remedy for headache?"*

Other multimodal approaches

Captioning

Describe each image in text, then use normal text search. What we built: simplest, works with any vector DB.

Shared embedding space

Models such as CLIP put images and text into the same vector space, so a text query can match an image directly.

Page-image retrieval

Embed screenshots of whole pages, so layout, tables and figures are searched together without extracting them.

Start with captioning; move to the others when captions lose important detail.

08

Putting it together

Choosing, debugging and practising



Which retriever, when?

Retriever	Strength	Weakness
Vector search	Understands meaning and paraphrases	Can miss exact rare terms and codes
BM25	Exact keywords, no model needed	Fails when wording differs
Hybrid (RRF / weighted)	Robust across both kinds of query	Two indexes to maintain
Hybrid + re-ranker	Best ordering of the final chunks	Extra latency per question
Multimodal	Finds facts in tables and images	Needs a captioning or vision model

Common problems and fixes

Symptom	Likely fix
The right chunk is never retrieved	Change chunking, increase k, add BM25 (hybrid)
Right chunk retrieved, but ranked low	Add a re-ranker
Answer contains made-up facts	"Only use the context" and "I don't know" rules; lower temperature
Questions about numbers or charts fail	Convert tables to sentences and caption images
Search is slow on large data	Use a vector database with an ANN index (HNSW)

Key takeaways

The LLM can only be as good as the context it is given

- Chunking decides what can be found
- Hybrid search covers meaning and exact words
- Tables and images become text to be searched
- Every vector DB is embed, store, compare
- Re-ranking puts the best chunk first